UNIVERSITÉ DE RENNES I
U.F.R. DE MATHÉMATIQUES

UNIVERSITÉ DE RENNES 1

Master 2 Mathématiques et applications
Parcours Calcul scientifique et modélisation
U.E. Logiciels Éléments Finis

Computation of the electromagnetic field induced in a conductor by a RF wave

Objective : to learn how to solve Maxwell's equations using Nédélec's Finite Elements and nodal Finite Elements with penalization.

Introduction

We consider a conducting dielectric object subject to a radio-frequency (RF) electromagnetic field. In such a situation, eddy currents are induced in the conductor and give rise to an induced RF field. The RF field is then modified in the vicinity of the ball. In this lab work we use FreeFem++ to compute this RF field perturbation.
For simplicity, the incident RF field $\mathbf{H_1}$ is assumed to be an electromagnetic field with harmonic variations with respect to time, uniform in space :

$$\mathbf{H_1}(t) = H_1 \, \mathrm{e}^{-\mathrm{i}\omega t} \, \mathbf{e_z}$$

where ω is the frequency and H_1 its constant complex amplitude. We denote by ε_1 the dielectric permitivity of the ball, by μ_1 its magnetic permeability and by σ_1 its conductivity. We assume that the surrounding medium is vacuum. Due to the linearity of Maxwell's equations in the situation considered here, the total magnetic field (sum of the incident and induced magnetic fields) is in the form

$$\mathbf{H}(\mathbf{x}, t) = \underline{\mathbf{H}}(\mathbf{x}) \, \mathrm{e}^{-\mathrm{i}\omega t}.$$

From Maxwell's equations, one can show that the amplitude $\underline{\mathbf{H}}$ of the magnetic field satisfies the following equations (in the sense of distribution) in all space

$$\mathrm{div}\,(\mu \underline{\mathbf{H}}) = 0, \tag{1}$$

$$\mathbf{curl}\left(\frac{1}{\sigma - \mathrm{i}\omega\varepsilon}\,\mathbf{curl}\,\underline{\mathbf{H}}\right) - \mathrm{i}\omega\mu\,\underline{\mathbf{H}} = 0. \tag{2}$$

In order to use the Finite Element Method, the computational domain must be bounded. We introduce an artificial boundary Γ far away from the dielectric conducting ball and we impose the following boundary condition on Γ :

$$\underline{\mathbf{H}}(\mathbf{x}) \wedge \mathbf{n} = H_1 \, (\mathbf{e_z} \wedge \mathbf{n}) \tag{3}$$

where $\mathbf{n}$ denotes the outward normal vector to the closed surface Γ. Such a boundary condition is justified by the decay in r^{-3}, where r denotes the radial distance from the origin taken inside the conduction, of the induced magnetic field. We denote by Ω the bounded open set in $\mathbb{R}^3$ delimited by Γ. Basically, Ω is the union of the conductor Ω_1 and the outer bounded domain Ω_0.
We introduce the following Sobolev's spaces for a Lipschitz domain $\Omega \in \mathbb{R}^3$

$$\begin{aligned}
\mathbb{H}^1(\Omega) &= \left\{\mathbf{u} \in \mathbb{L}^2(\Omega)^3 \mid \forall i, j \in \{1,2,3\}\ \partial_j u_i \in \mathbb{L}^2(\Omega)\right\} \\
\mathbb{H}(\mathbf{curl}\,, \Omega) &= \left\{\mathbf{u} \in \mathbb{L}^2(\Omega)^3 \mid \mathbf{curl}\,(\mathbf{u}) \in \mathbb{L}^2(\Omega)^3\right\} \\
\mathbb{H}_0(\mathbf{curl}\,, \Omega) &= \left\{\mathbf{u} \in \mathbb{H}(\mathbf{curl}\,, \Omega) \mid \mathbf{u} \wedge \mathbf{n} = \mathbf{0} \text{ on } \Gamma\right\}
\end{aligned}$$

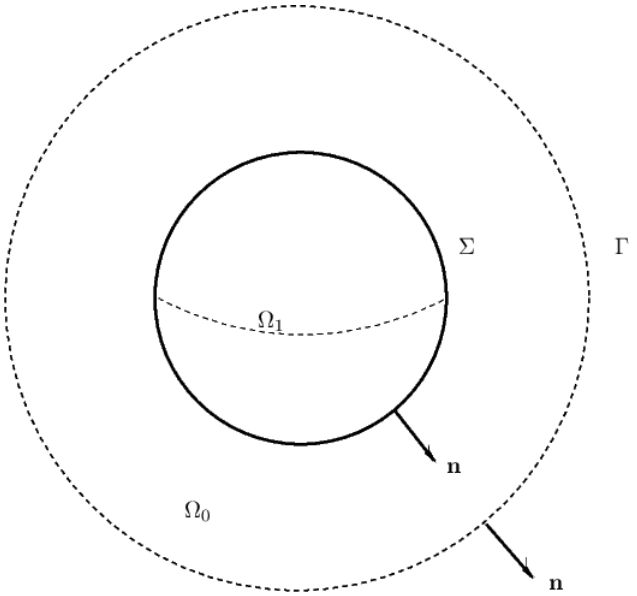


FIGURE 1 – Situation considered and notations used.

and we recall the Green's formula for the curl operator in 3D

$$\forall \mathbf{u} \in \mathbb{H}(\mathbf{curl}\,,\Omega)\ \forall \mathbf{v} \in \mathbb{H}^1(\Omega) \qquad \int_\Omega \mathbf{curl}\,(\mathbf{u}) \cdot \mathbf{v}\,\mathrm{d}x - \int_\Omega \mathbf{curl}\,(\mathbf{v}) \cdot \mathbf{u}\,\mathrm{d}x = \oint_{\partial\Omega} \mathbf{u} \cdot \big(\mathbf{v} \wedge \mathbf{n}\big)\,\mathrm{d}\sigma. \tag{4}$$

The space $\mathbb{H}(\mathbf{curl}\,,\Omega)$ will be the appropriate Sobolev's space for a weak formulation of our problem.

1 Using Green's formula (4), show that the weak formulation for the boundary value problem (2)–(3) reads : Find $\underline{\mathbf{H}} \in \mathbb{H}(\mathbf{curl}\,,\Omega)$ such that $\operatorname{div} \underline{\mathbf{H}} = 0$ in Ω_0 and in Ω_1, $\underline{\mathbf{H}}(\mathbf{x}) \wedge \mathbf{n} = H_1\,(\mathbf{e_z} \wedge \mathbf{n})$ on Γ and such that for all $\mathbf{v} \in \mathbb{H}_0(\mathbf{curl}\,,\Omega)$

$$\int_\Omega \frac{1}{\sigma - \mathrm{i}\omega\varepsilon}\ \mathbf{curl}\,\underline{\mathbf{H}} \cdot \mathbf{curl}\,(\mathbf{v})\,\mathrm{d}x - \mathrm{i}\omega \int_\Omega \mu\ \underline{\mathbf{H}} \cdot \mathbf{v}\,\mathrm{d}x = 0 \tag{5}$$

where μ, ε and σ denote piece-wise constant functions respectively equal to μ_1, ε_1 and σ_1 in Ω_1 and μ_0, ε_0 and 0 in the outer domain Ω_0.

Approximation by Lagrange Finite Elements

We may want to compute the solution to the weak formulation (5) under FREEFEM++ using Lagrange Finite Elements. We consider as an example a copper ball. Dielectric permittivity of conductors is equal to the permittivity of free space and their magnetic permeability is also the one of free space.

The first step is to generate a mesh of the whole domain Ω corresponding to the union of Ω_0 and Ω_1 that preserves the boundary Σ (*i.e.* no element must cross the surface Σ). In Appendix A, we explain how to get with FREEFEM++ a three-dimensional mesh using the mesh generator `tetgen`. The reader is referred to this appendix for details on the way to generate the 3-D mesh of Ω.

The FREEFEM++ program given below allows the computation of the solution $\mathbf{H}$ to the weak formulation (5) by Lagrange P_1 Finite Element.

```
load "msh3"
load "tetgen"
load "medit"
include "mesh_sphere.idp"  // file containing the function for generating the mesh of a sphere
//
// Physical data of the problem
```

```
//
real mu0=4*pi*1E-7; // vacuum magnetic permeability [H/m]
real eps0=8.854E-12; // electric permittivity of vaccum [F/m]
real sigma1=59E6; // electric conductivity of copper [S/m]
real eps1=eps0; //Permittivity of conductors is equal to the permittivity of free space
real B1=1E-4;  // RF field strength [Tesla]
real H1=B1/mu0;
real omega=42.58E6; // frequency in [Hz]
//
// Mesh of the 3D domain
//
real unit=1E-2; // normalization : distances are given in cm (1cm=1E-2 m)
real R=1*unit, Re=4*unit; // inner and outer radius of the hollow ball
real hi=0.2*unit, he=0.4*unit; // mesh step-size on the inner and outer spheres

meshS ThSi=Sphere(R,hi,11,1); // meshes the inner sphere
meshS ThSe=Sphere(Re,he,12,1); // meshes the outer sphere

meshS ThS=ThSi+ThSe; // the 2 surfacic meshes are gather together

real volteti=hi^3/6, voltete=he^3/6; // max allowed volume for tetrahedrons as required by Tetgen
real[int] Pi=[0,0,0], Pe=[0,0,(Re+R)/2]; // point belonging to the inner domain
real[int] domaine=[Pi,1,volteti,Pe,2,voltete]; // point belonging to the outer domain
mesh3 Th=tetg(ThS,switch="paAAYYQ",nbofregions=2,regionlist=domaine);
//
// Finite Element computation
//
fespace Vh(Th,[P1,P1,P1]);
Vh<complex> [Hx,Hy,Hz],[Vx,Vy,Vz];

macro Curl(Hx,Hy,Hz) [dy(Hz)-dz(Hy),dz(Hx)-dx(Hz),dx(Hy)-dy(Hx)] // EOM

solve rfmag([Hx,Hy,Hz],[Vx,Vy,Vz])=
      int3d(Th,1)( (1/(1i*eps1*omega-sigma1))*Curl(Hx,Hy,Hz)'*Curl(Vx,Vy,Vz))
    + int3d(Th,2)( (1/(1i*eps0*omega))*Curl(Hx,Hy,Hz)'*Curl(Vx,Vy,Vz))
    + int3d(Th)(1i*omega*mu0*[Hx,Hy,Hz]'*[Vx,Vy,Vz])
+ on(12,Hx=0,Hy=0,Hz=H1);
```

Attention should be given to the version of FREEFEM++ used. The above program is for versions of FREEFEM++ newer that release 4.6. For older versions, one should replace the following lines in the code given above

```
meshS ThSi=Sphere(R,hi,11,1); // meshes the inner sphere
meshS ThSe=Sphere(Re,he,12,1); // meshes the outer sphere
meshS ThS=ThSi+ThSe; // the 2 surfacic meshes are gather together
```

by

```
mesh3 ThSi=Sphere(R,hi,11,1); // meshes the inner sphere
mesh3 ThSe=Sphere(Re,he,12,1); // meshes the outer sphere
mesh3 ThS=ThSi+ThSe; // the 2 surfacic meshes are gather together
```

2 Append this FREEFEM++ program to compute the divergence of the solution $\underline{\mathbf{H}}$. Plot the euclidean norm of $\operatorname{div} \underline{\mathbf{H}}$ in the computational domain and compute the L^2 norm of $\operatorname{div} \underline{\mathbf{H}}$. What can we conclude ?

Comparison with the exact solution

An exact solution to the PDE problem (1)– (2) is given by [1]

$$\mathbf{H}(\mathbf{r}) = \begin{cases} H_1\,\beta\left(\dfrac{f'(r)}{r} + k^2 f(r)\right)\mathbf{e_z} - H_1\,\beta\,\dfrac{z}{r^2}\left(3\dfrac{f'(r)}{r} + k^2\,f(r)\right)\mathbf{r} & \text{in } \Omega_1 \\ \dfrac{4\pi R^3\alpha}{3r^3}\left(\dfrac{3H_1 z}{r^2}\,\mathbf{r} - H_1\,\mathbf{e_z}\right) & \text{in } \Omega_0 \end{cases}$$

where $\mathbf{r} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$, $r = \sqrt{x^2+y^2+z^2}$, $f(r) = \dfrac{\sin(kr)}{r}$, $k = \dfrac{1+\mathrm{i}}{\delta}$, and $\delta = \sqrt{\frac{2}{\omega\mu_1\sigma_1}}$ is referred in the literature as the skin depth,

$$\begin{aligned} \beta &= \frac{3\mu_0 R^3}{2D} \\ \alpha &= -\frac{3}{8\pi D}\left(\mu_0(kR)\cos(kR) - \mu_0\sin(kR) + \mu_0(kR)^2\sin(kR) + 2\mu_1(kR)\cos(kR) - 2\mu_1\sin(kR)\right) \\ D &= \mu_0(kR)\cos(kR) - \mu_0\sin(kR) + \mu_0(kR)^2\sin(kR) - \mu_1(kR)\cos(kR) + \mu_1\sin(kR) \end{aligned}$$

3 Compute the relative error in the $\mathbb{L}^2$ norm made when solving problem (1)– (2) by the Finite Element method using Lagrange Finite Element.

Nédélec's Finite Element [2]

Several difficulties have been identified when Lagrange finite elements are employed to compute vector electromagnetic fields. Among the more serious are the continuity requirements for the approximated vector fields that cannot be easily fulfilled as well as the difficulty in applying of the boundary conditions. To overcome these obstacles the so-called *edge elements* (or vector finite elements) are used. They are characterized by the fact that the degree of freedom are assigned to the edges of an element (instead of the vertexes and other nodes for Lagrange finite elements). The edge finite element is geometrically the same as its nodal counterpart. The edge element has one shape function for each of the edges. Edge finite element method has several advantages in 3-D electromagnetic field computations compare to nodal finite elements. Edge finite elements satisfy continuity of only tangential or normal field components across interfaces between two adjacent finite elements. Thus edge finite element approximation does not impose any additional constraints on the approximated field apart from those prescribed by the nature of the field itself.
Among the edge finite elements available under FREEFEM++ is the so-called "Edge03d" finite elements corresponding to Nédélec finite elements well suited to approximate the $\mathbb{H}(\mathbf{curl}\,,\Omega)$ space.

4 Use Nédélec finite element to compute with FREEFEM++ an approximation of the solution to the weak formulation (5) of our PDE problem.

5 Compute the divergence of $\mathbf{H}$ in the computational domain and its $\mathbb{L}^2$ norm. Compute the relative error in the $\mathbb{L}^2$ norm made when solving problem (1)– (2) by the Finite Element method using Nédélec Finite Element.

Lagrange Finite Element with penalization

The above computations have revealed that nodal Finite Elements are inappropriate for Finite Element computations in electromagnetism due to the non preserving of the divergence free condition for the magnetic field ($\mathrm{div}\,\underline{\mathbf{H}} = 0$). One way to overcome the difficulty is to introduce in the weak formulation

1. See L.D. Landau and E.M. Lifshitz, Electrodynamics of Continuous Media (Volume 8 of A Course of Theoretical Physics), Pergamon Press, 1960.

2. Jean-Claude Nédélec is Professor Emeritus at University of Rennes 1 from September 2008, `http://www.cmap.polytechnique.fr/~nedelec`.

a penalization term involving the divergence of $\underline{\mathbf{H}}$. Namely, the weak formulation (5) is changed into the following one : Find $\underline{\mathbf{H}} \in \mathbb{H}(\mathbf{curl}\,, \Omega)$ such that $\operatorname{div} \underline{\mathbf{H}} = 0$ in Ω_0 and in Ω_1, $\underline{\mathbf{H}}(\mathbf{x}) \wedge \mathbf{n} = H_1\,(\mathbf{e_z} \wedge \mathbf{n})$ on Γ and such that for all $\mathbf{v} \in \mathbb{H}_0(\mathbf{curl}\,, \Omega)$

$$\int_\Omega \frac{1}{\sigma - \mathrm{i}\omega\varepsilon}\ \mathbf{curl}\,\underline{\mathbf{H}} \cdot \mathbf{curl}\,(\mathbf{v})\,\mathrm{d}x + \alpha \int_\Omega \operatorname{div} \underline{\mathbf{H}}\ \operatorname{div}(\mathbf{v})\,\mathrm{d}x - \mathrm{i}\omega \int_\Omega \mu\ \underline{\mathbf{H}} \cdot \mathbf{v}\,\mathrm{d}x = 0 \tag{6}$$

where μ, ε and σ denote piece-wise constant functions respectively equal to μ_1, ε_1 and σ_1 in Ω_1 and μ_0, ε_0 and 0 in the outer domain Ω_0. The value of the "penalization parameter" α has to be appropriately chosen.

6 Compare this approach to the use of Nédélec Finite Element to compute with FREEFEM++ an approximation of the solution to the weak formulation of our PDE problem.

Appendix A : 3D meshing of a hollow ball under FREEFEM++

In this appendix, we explain how to get with FREEFEM++ a three-dimensional mesh of the domain Ω that preserves the two domains Ω_0 and Ω_1. In the particular case considered here, the problem turns out to meshing a hollow ball.

Three dimensional meshes with `TetGen` under FREEFEM++

FREEFEM++ includes several mesh generators. One of these mesh generators is `TetGen`. `TetGen` is a program to generate tetrahedral meshes of any 3D polyhedral domains. [3] We will see in the following how to use `TetGen` to generate a mesh of a hollow ball. From a practical point of view, in order to build a mesh of a three dimensional domain with `TetGen`, one has to provide to `TetGen` a mesh of the outer boundary surface of the domain. This can be done by using a parametrization Φ of the surface. We mesh the domain of the parametrization function and then we move this mesh onto the surface by computing the image of the mesh triangles vertexes by Φ. From a practical point of view this can be achieved by the directive **movemesh23** of the `msh3` library of FREEFEM++. The `msh3` library has to be loaded in memory for the directive **movemesh23** to be available by starting the FREEFEM++ program with

```
load "msh3"
```

For instance, a parametrization of the sphere centered at the origin of the coordinate system with radius R is :

$$\Phi : (\theta, \varphi) \in [0, \pi] \times [0, 2\pi] \longmapsto \begin{pmatrix} R\sin(\theta)\cos(\varphi) \\ R\sin(\theta)\sin(\varphi) \\ R\cos(\theta) \end{pmatrix},$$

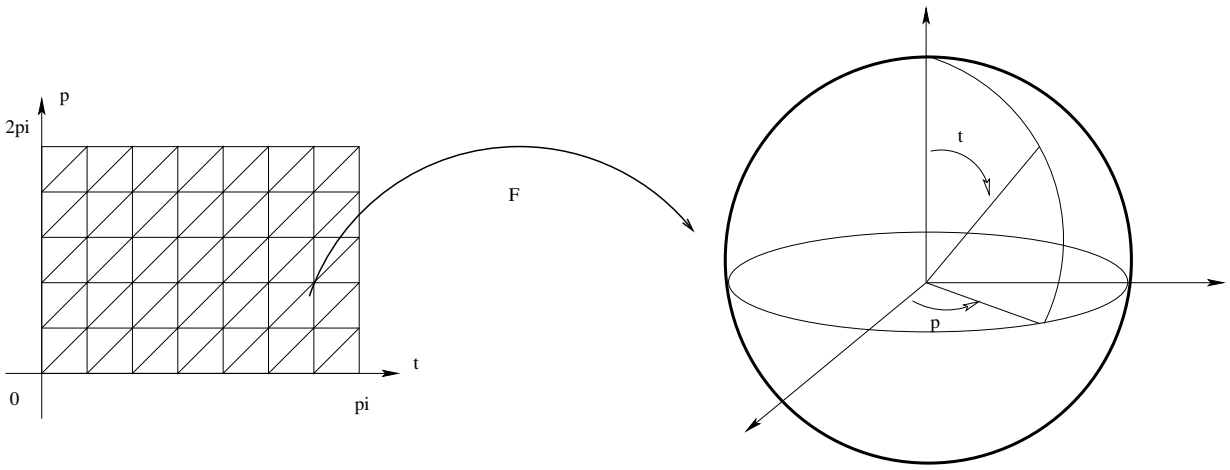


FIGURE 2 – Parametrization of a sphere.

A mesh of the sphere is obtained under FREEFEM++ as follows.

```
load "msh3"  //  Library for 3D meshes
load "medit"  // Medit library for 3D meshes visualization
load "tetgen" // Library for Tetgen mesh generator

real R=1; // radius of the sphere
// mesh of the parametrization domain [0,\pi]\times[0,2\pi]
int n=20; // number of nodes in each direction
mesh Th1=square(n,n,[pi*x,2*pi*y]);
plot(Th1);
```

3. `TetGen` has been developped by Hang Si from the Weierstrass Institute for Applied Analysis and Stochastics, Berlin, Germany, see `http://wias-berlin.de/software/tetgen/` for details.

```
// mesh of the surface obtained by moving the parametrization domain mesh onto the sphere
meshS ThS=movemesh23(Th1,transfo=[R*sin(x)*cos(y),R*sin(x)*sin(y),R*cos(x)]);
medit("Maillage de la sphere",ThS); // plot the mesh
```

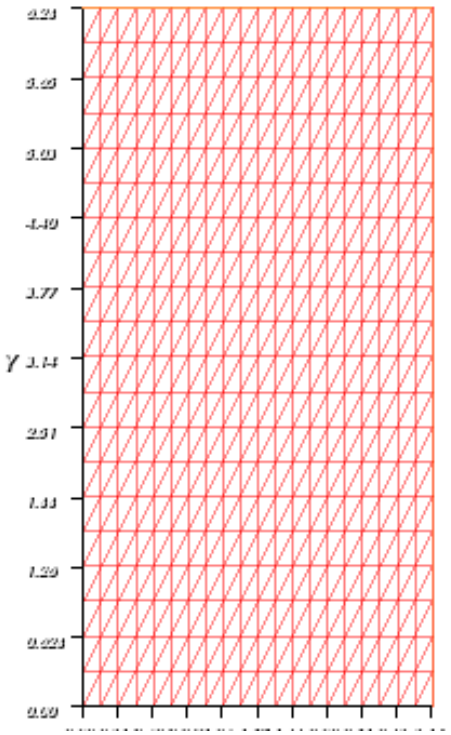
Attention should be given to the version of FREEFEM++ used. The above program is for versions of FREEFEM++ newer that release 4.6. For older versions, one should replace the following lines in the code given above

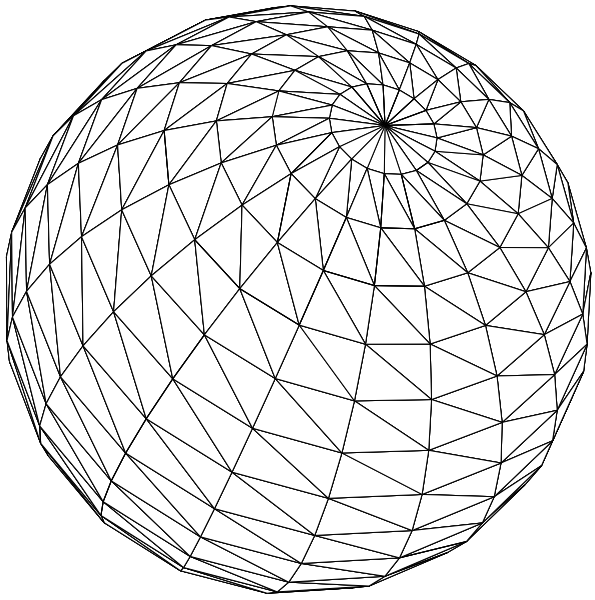
```
meshS ThS=movemesh23(Th1,transfo=[R*sin(x)*cos(y),R*sin(x)*sin(y),R*cos(x)]);
```

by

```
mesh3 ThS=movemesh23(Th1,transfo=[R*sin(x)*cos(y),R*sin(x)*sin(y),R*cos(x)]);
```

We obtain the following meshes for the parametrization function domain and parametrization function image (the sphere).





We trigger the mesh generation of the volume enclosed by the sphere with `TetGen` by the following instruction :

```
mesh3 ThB=tetg(ThS);
```

Unfortunately, due to the mesh degeneration in the north and south poles area, the mesh generator `TetGen` may not succeed in creating the mesh of the volume enclosed by the sphere with tetrahedrons. Moreover, even if `TetGen` provides a mesh of the domain enclosed in the sphere, the tetrahedrons are likely to be "degenerated" around the pole of the sphere.

To circumvent this issue one can distort the mesh of the parametrization domain $[0, \pi] \times [0, 2\pi]$ so that when moved on the sphere thanks to the parametrization function Φ, the elements around the two poles (corresponding to $\theta = 0$ and $\theta = \pi$) have a "regular" shape. We can realize this mesh adaptation by using the directive **adaptmesh**. The directive **adaptmesh** creates a new mesh from a given one adapted to the Hessian of a function f (formula or FE-function) or adapted according to a metric provided by the user. The first case correspond to the situation where mesh adaptation is mostly used to increase the accuracy of

the solution of a problem when it varies locally and sharply. The mesh is refined where the solution varies the most in order to "capture" the variations of the solution through a finer grid. Here we are concerned by the second usage where mesh adaptation is used for a different goal : we want that the distance between any two vertexes of a triangle in the mesh of the sphere remains roughly constant.
We define a new metric (the mathematical tool to measure distances) and replace the Euclidean space by a metric space where the metric is defined by

$$d_{\mathcal{M}}(P,Q) = \sqrt{\mathbf{PQ}^\top \ \mathcal{M} \ \mathbf{PQ}}$$

where $\mathcal{M}$ denotes a symmetric positive definite matrix. [4] We then distort the initial mesh according to this new metric so that once projected on the sphere surface the property that the distance between any 2 vertexes of a mesh's triangle remains roughly constant. Here we choose a matrix $\mathcal{M}$ adapted to the measure of distances in the sphere. It is constructed from the spherical coordinates parametrization function of a sphere. The way to proceed with the directive **adaptmesh** is detailed in the following example.

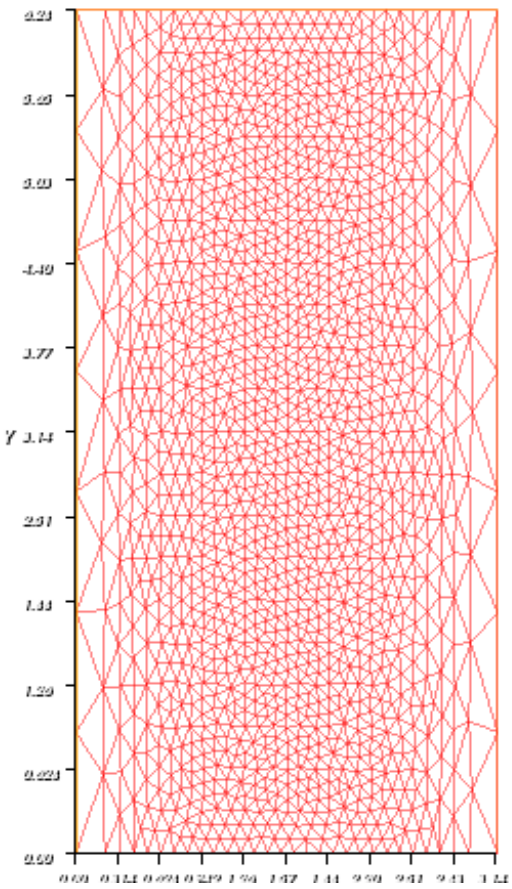
```
mesh Th2=square(n/2,n,[x*pi,2*y*pi]);// mesh of the parametrization domain [0,\pi]\times[0,2\pi]
// Parametrization function
func f1=R*sin(x)*cos(y);
func f2=R*sin(x)*sin(y);
func f3=R*cos(x);
// Partial derivatives of the parametrization function
func f1x=R*cos(x)*cos(y);
func f1y=-R*sin(x)*sin(y);
func f2x=R*cos(x)*sin(y);
func f2y=R*sin(x)*cos(y);
func f3x=-R*sin(x);
func f3y=0;
// definition of the metric tensor
func m11=f1x^2+f2x^2+f3x^2;
func m21=f1x*f1y+f2x*f2y+f3x*f3y;
func m22=f1y^2+f2y^2+f3y^2;

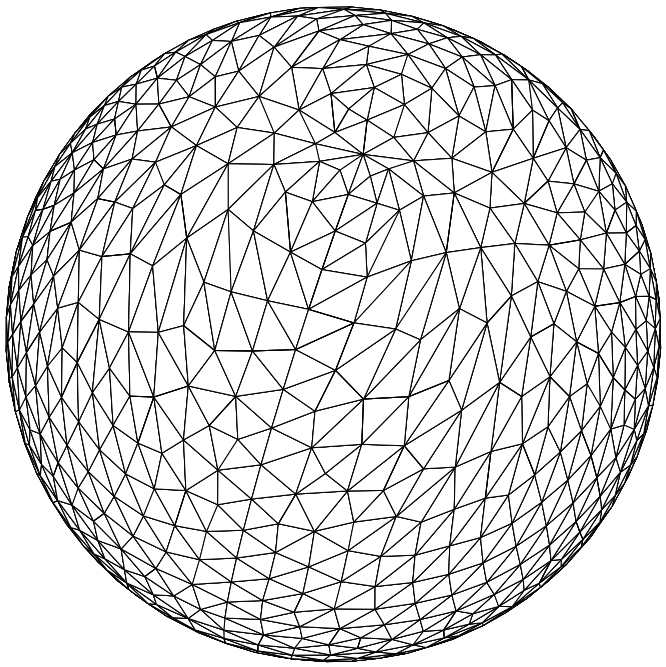
func perio=[[4,y],[2,y],[1,x],[3,x]];// in order to have a symmetric mesh
real h=0.1;// mesh step-size on the sphere
real vv= 1/square(h);
plot(Th2);
Th2=adaptmesh(Th2,m11*vv,m21*vv,m22*vv,IsMetric=1,periodic=perio);
plot(Th2);

int[int] ref=[0,10];  // The reference number 10 is assigned to the tetrahedron face
                                  // located on the sphere
meshS ThS2= movemesh23(Th2,transfo=[f1,f2,f3],reftri=ref);
mesh3 Th=tetg(ThS2);
medit("Maillage de la boule",Th); //  Plot the mesh
```

The new mesh of the parametrization domain and the new mesh of the sphere are depicted bellow.

4. For details on what is a metric of mesh adaption in FreeFem++, we refer to the following document `http://www.freefem.org/ff2a3/ff++days-2010/exposes/F-Hecht-Metric.pdf`.





Attention should be given to the version of FREEFEM++ used. The above program is for versions of FREEFEM++ newer that release 4.6. For older versions, one should replace the following lines in the code given above

```
meshS ThS2= movemesh23(Th2,transfo=[f1,f2,f3],reftri=ref);
```

by

```
mesh3 ThS2= movemesh23(Th2,transfo=[f1,f2,f3],refface=ref);
```

Then, one uses `TetGen` to generate the mesh of the volume enclosed by the sphere with the new mesh of the sphere as in the previous program example.

In order to mesh a hollow ball by the above method with `TetGen`, it is necessary to generate the mesh of both the outer and inner sphere. We can make the meshing process simpler by writing a FREEFEM++ function that generate the mesh of a sphere with given radius. The function will be called two times. We have gather together in the function `Sphere` the FREEFEM++ instructions that generates a mesh of a sphere.

```
func meshS Sphere(real R, real h, int L, int orient)
{
// Generate the mesh of a sphere centered at the coordinates origin with radius R
// Parameters IN :
// R = sphere radius
// h = step-size of the mesh of the sphere
// L = reference number assigned to the face of the tetrahedron located on the sphere
// orient = 1 or -1 according to the orientation
//
```

```
// Parameter OUT:
// mesh of the sphere

int n = 20; // nb of vertex on each side of the 2D parametrization domain
mesh Th=square(n/2,n,[x*pi,2*y*pi]);// mesh of the parametrization domain [0,\pi]\times[0,2\pi]
// Parametrization function
func f1=R*sin(x)*cos(y);
func f2=R*sin(x)*sin(y);
func f3=R*cos(x);
// Partial derivatives of the parametrization function
func f1x=R*cos(x)*cos(y);
func f1y=-R*sin(x)*sin(y);
func f2x=R*cos(x)*sin(y);
func f2y=R*sin(x)*cos(y);
func f3x=-R*sin(x);
func f3y=0;
// definition of the metric tensor
func m11=f1x^2+f2x^2+f3x^2;
func m21=f1x*f1y+f2x*f2y+f3x*f3y;
func m22=f1y^2+f2y^2+f3y^2;

func perio=[[4,y],[2,y],[1,x],[3,x]];// in order to have a symmetric mesh
real vv= 1/square(h);
Th=adaptmesh(Th,m11*vv,m21*vv,m22*vv,IsMetric=1,periodic=perio);

int[int] ref=[0,L];  // modification of the reference numbers 0 --> L
meshS ThS= movemesh23(Th,transfo=[f1,f2,f3],reftri=ref,orientation=orient);
return ThS;
}
```

Attention should be given to the version of FREEFEM++ used. The above program is for versions of FREEFEM++ newer that release 4.6. For older versions, one should replace the following lines in the code given above

```
meshS ThS= movemesh23(Th,transfo=[f1,f2,f3],reftri=ref,orientation=orient);
```

by

```
mesh3 ThS= movemesh23(Th,transfo=[f1,f2,f3],refface=ref,orientation=orient);
```

Moreover, the function definition must be

```
func mesh3 Sphere(real R, real h, int L, int orient)
```

instead of

```
func meshS Sphere(real R, real h, int L, int orient)
```

This FREEFEM++ function must be saved in a file with the extension `.idp` (for instance `mesh_sphere.idp`) to be used latter on in a FREEFEM++ program.

A mesh of the hollow ball is then obtained by 2 calls to the `mesh_sphere` function as follows :

```
load "msh3"
load "medit"
load "tetgen"
include "mesh_sphere.idp"

real Ri=1, Re=3; // inner and outer radii of the hollow ball
```

```
real hi=0.1, he=0.5; // step-size of the meshes of the inner and outer sphere
meshS ThSi=Sphere(Ri,hi,11,1); //  mesh the inner sphere
meshS ThSe=Sphere(Re,he,12,1); // mesh the outer sphere
meshS ThS=ThSi+ThSe; // the 2 surface meshes are joined
```

For version of FreeFem++ older than 4.6, the type `meshS` should be changed for `mesh3`.

```
real volteti=hi^3/6, voltete=he^3/6; // maximum volume of the tetrahedrons
real[int] Pi=[0,0,0], Pe=[0,0,(Re+Ri)/2]; // points in the inner and outer domains
real[int] domaine=[Pi,1,volteti,Pe,2,voltete]; // properties of the 2 domains
mesh3 Th=tetg(ThS,switch="paAAYYQ",nbofregions=2,regionlist=domaine);
medit("2 boules concentriques",Th);
```

The various options available for the parameter `switch` in the call to `TetGen` are detailed in the FreeFem++ documentation. In the above example, they are the following.

- the `p` letter requires a tetrahedralization of the boundary
- the `a` letter indicates a constraint on the volume of the tetrahedrons (given in the array of the `regionlist` keyword),
- the 2 letters `AA` indicate that each domain will be given a distinct reference number (given in the array of the `regionlist` keyword),
- the letter `Y` indicates that a conform mesh on both side of the inner boundary is required,
- the letter `Q` indicates that the execution must be done in quiet mode (no warning messages).

The parameter `regionlist` gets an array of real numbers corresponding to the list of : the coordinates of one point belonging to each sub-domain to be meshed, the reference number of the sub-domain and the maximal allowed volume for the tetrahedrons. This list is repeated for each sub-domain.

Note that among the other mesh generators available under FreeFem++, one of them, reachable through the directive **buildlayers**, builds three-dimensional meshes by piling up layers obtained from a bi-dimensional mesh. But this approach is not well suited for generating a mesh of a sphere. It is best suited for cylindric geometries.